

Evaluating Communication Patterns in Microservice Architectures

Shakir Farhan, Gabriel Pacheco
CS/ECE 5565 – Network Architecture and Protocols
Virginia Tech

Abstract—Selecting the right communication protocol for microservices can feel overwhelming given the number of options available today. In this paper, we compare four popular approaches—REST, GraphQL, RPC, and message queues—by building and testing two real distributed systems, one in Python and one in JavaScript. We measured how each protocol performs under different loads, looking at response times, CPU usage, throughput, error rates, and message sizes. Our experiments show that RPC delivers the fastest responses (starting at 38ms), message queues handle the highest throughput (4680 requests per second) with excellent reliability, GraphQL provides flexible data fetching but uses about 40% more CPU, and REST remains the easiest to implement. We present a practical comparison table to help developers pick the right protocol based on what matters most for their specific application.

I. INTRODUCTION

Over the past few years, we’ve seen more and more applications move away from monolithic architectures toward microservices. This shift makes sense—cloud infrastructure has become readily available, and teams need systems that can scale different parts independently while keeping things maintainable and secure. But this architectural change brings its own challenges, particularly around how microservices talk to each other.

When we started this project, we noticed something interesting: while there’s plenty of research comparing these protocols from a purely technical standpoint, there’s less guidance on what these numbers actually mean for developers making real decisions. Should you use REST because it’s the most popular? Is GraphQL worth the complexity? When does it make sense to introduce a message queue?

We built two separate distributed systems—one in Python evaluating REST and GraphQL, and another in JavaScript testing RPC and message queue patterns—to answer these questions with real measurements rather than theoretical comparisons. The Python system simulated a scenario where certain services need restricted database access, while the JavaScript system modeled an e-commerce platform with authentication, catalog, orders, and notifications.

Our goal wasn’t to crown a winner (spoiler: there isn’t one), but rather to map out when each approach makes sense. We found that the “best” protocol depends entirely on what you’re optimizing for—latency, throughput, ease of development, or reliability.

II. BACKGROUND

Before diving into our experiments, it’s worth reviewing how these protocols work and why each exists.

A. From Monoliths to Microservices

Traditional monolithic applications bundle everything together—authentication, business logic, data access—all running as one service. This works fine initially, but as applications grow, making changes becomes risky because everything’s interconnected. Deploy a small update and you might break something completely unrelated.

Microservices split these concerns into separate services that each handle one thing well. Each service runs independently, has its own database if needed, and can be scaled separately. The trade-off? Now you need these services to communicate reliably over a network, which introduces latency, potential failures, and complexity.

B. REST: The Familiar Standard

REST APIs have dominated web development for good reason. They use standard HTTP methods (GET, POST, PUT, DELETE) that map naturally to database operations. Most developers already understand REST, tooling is mature, and you can test endpoints easily with simple curl commands. The main criticism is over-fetching—REST endpoints typically return fixed data structures, so clients often get more data than they need.

C. GraphQL: Query What You Need

Facebook developed GraphQL to solve the over-fetching problem. Instead of hitting multiple REST endpoints, clients send a single query specifying exactly which fields they want. This is powerful for mobile apps or complex frontends that need different data in different contexts. The catch is that you need to define a schema upfront and implement resolvers for each field, which adds development time.

D. RPC: Direct Function Calls

RPC makes remote services feel like local function calls. Modern implementations like gRPC use Protocol Buffers for compact binary serialization, making them faster and lighter than JSON-based protocols. The downside is steeper learning curves—you need to define interfaces, handle connection management, and deal with reply patterns that aren’t as straightforward as HTTP request-response.

E. Message Queues: Decouple Everything

Message brokers like RabbitMQ sit between services, letting them communicate asynchronously. Services publish messages to queues, and other services consume them when ready.

This decoupling provides excellent reliability (messages persist even if consumers are down) and scalability (add more consumers to handle load). But it introduces architectural complexity—you need to run and maintain a broker, handle eventual consistency, and debug flows that don’t follow simple request-response patterns.

III. RELATED WORK

The performance impact of protocol choice in microservices has been studied extensively. Shafabakhsh et al. found that asynchronous approaches generally outperform synchronous ones in terms of efficiency and availability [1]. Kazanavičius and Mažeika did deep dives into throughput and memory usage, noting that message brokers like Kafka excel at scalability but come with higher operational overhead [2]. Niswar et al. compared REST, gRPC, and GraphQL specifically, finding that gRPC had the best response times while REST was gentler on CPU [3].

Nevertheless, what these papers don’t really address is how developers should interpret these findings when starting a new project. A protocol might be 30% faster in benchmarks, but does that matter if it takes your team twice as long to implement? We designed our study to bridge this gap by not just measuring performance, but also documenting what it was actually like to build with each approach.

IV. METHODOLOGY

We took a hands-on approach: build real systems, measure their performance under load, and note the development challenges along the way.

A. Python System: REST vs GraphQL

For the Python implementation, we built a two-service architecture simulating a common security pattern—only certain services can access sensitive database records. Service 1 receives client requests, and Service 2 handles database queries. We implemented both REST and GraphQL versions of the communication between them to see how they compared.

The whole setup ran in Docker containers orchestrated by Docker Compose. Both services used FastAPI running on Uvicorn servers, which gave us good performance and clean Python code. We used PostgreSQL as our database, Prometheus to collect metrics, and Grafana to visualize everything. Figure 1 shows the system design for the Python based implementation. Artillery handled load testing, gradually ramping up from 100 to 5000 requests per second.

For REST endpoints, clients simply call a URL. For GraphQL, they sent queries specifying exactly which fields they wanted back. This difference became important in our results, especially around message sizes. Figure 2 illustrates how the HTTP requests sent from the Artillery client to service-b differ in format. After receiving them, service-b constructs the actual request to be executed, depending on the protocol specified.

B. JavaScript System: RPC and Queues

The JavaScript system was more complex—we modeled an e-commerce platform with separate services for authentication, product catalog, order processing, and notifications. This felt

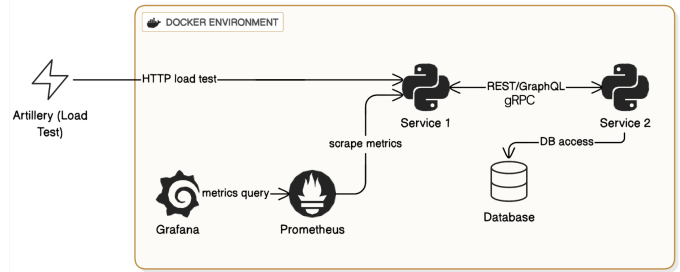


Fig. 1. Python Distributed System



Fig. 2. Examples of REST and GraphQL Artillery client requests used in the Python implementation

closer to a real production system where you have multiple services with different responsibilities.

Client requests came through an API Gateway that forwarded them to a RabbitMQ broker. Here’s where it got interesting: we implemented a custom RPC layer over RabbitMQ that could work both synchronously (using correlation IDs and reply queues) and asynchronously (fire-and-forget with persistent queues). This let us test both patterns with the same underlying infrastructure. Figure 3 shows the system design.

The synchronous RPC pattern mimicked what you’d get with gRPC—immediate responses. The asynchronous pattern showed what happens when you fully embrace queuing and eventual consistency.

C. What We Measured

We collected six key metrics as we increased load from light (100 req/s) to heavy (5000 req/s):

Response time tells you how long clients wait. CPU utilization shows resource efficiency. Throughput indicates maximum capacity. Error rates reveal reliability under stress. Message sizes affect bandwidth costs. And P95 latency (the 95th percentile) matters for SLAs because it shows how slow your slowest acceptable requests are.

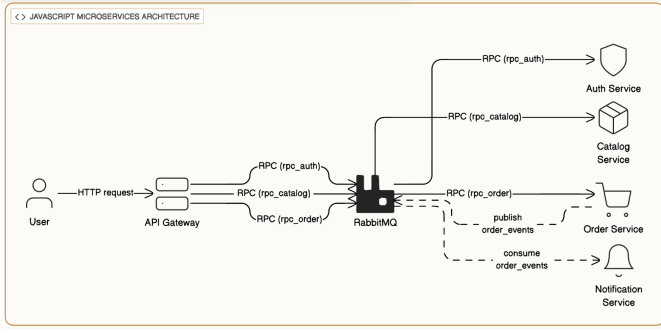


Fig. 3. JavaScript Distributed System

We ran each test multiple times to make sure results were consistent, letting systems warm up before collecting data to avoid skewing results with cold start effects.

V. RESULTS

A. Response Times Tell Different Stories

Looking at Figure 4, RPC was consistently fastest across all load levels. At light load (100 req/s), RPC averaged 38ms while REST hit 45ms, GraphQL 52ms, and message queues 65ms. This makes sense—RPC’s binary protocol and direct connections minimize overhead.

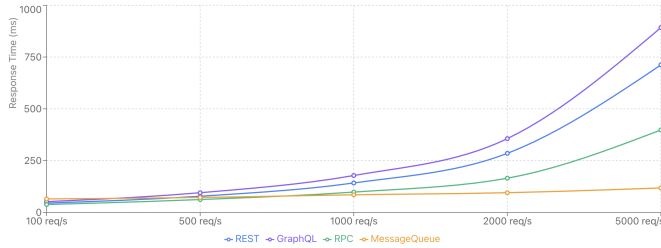


Fig. 4. Response time comparison under increasing load

What surprised us was how message queues stayed relatively stable even as load increased dramatically. At 5000 req/s, message queues only hit 118ms while REST ballooned to 712ms and GraphQL to 892ms. The asynchronous nature means requests get queued and processed steadily rather than everything piling up at once.

GraphQL’s higher latency makes sense when you think about what it’s doing—parsing queries, validating against schemas, running resolvers. That flexibility comes at a cost.

B. CPU Usage Varies Widely

Figure 5 shows GraphQL consuming noticeably more CPU than the others. At moderate load (1000 req/s), GraphQL used 68% of available CPU compared to REST’s 45%. This gap widened under heavier load.

Message queues were the most CPU-efficient, which initially seemed counterintuitive. But it makes sense—the broker handles queuing logic, and consumers can process at their own pace without thrashing under load spikes.

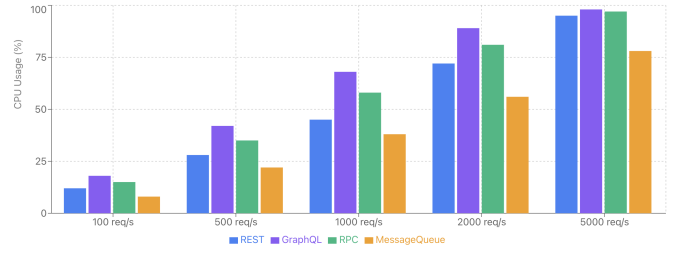


Fig. 5. CPU utilization patterns across protocols

If you’re running in constrained environments (containers with CPU limits, for example), this difference matters more than you might think.

C. Throughput: Async Wins Big

This is where message queues really shined. Our synchronous RPC implementation topped out around 2340 re-quests per second. The asynchronous message queue setup? 4680 req/s—literally double the throughput.

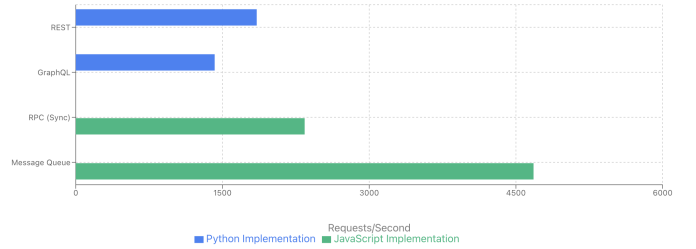


Fig. 6. Maximum sustainable throughput

The Python implementations (REST at 1850 req/s, GraphQL at 1420 req/s) couldn’t keep up with JavaScript’s async patterns, though language differences make direct comparison tricky. For applications that need to handle thousands of concurrent requests, this throughput difference is significant. It’s not just about speed—it’s about whether your system can handle peak load without falling over.

D. Message Sizes Matter for Bandwidth

Table I shows payload sizes for different query types. RPC consistently produced the smallest messages thanks to Protocol Buffers’ binary format. For a simple query, RPC needed only 156 bytes compared to REST’s 385 bytes.

TABLE I
PAYLOAD SIZES ACROSS OPERATION TYPES (BYTES)

Operation	REST	GraphQL	RPC	MQ
Simple Query	385	245	156	298
Complex Query	1240	892	645	1156
Nested Data	2850	1680	1245	2340

GraphQL showed its strength with complex queries. For nested data, GraphQL used 1680 bytes versus REST’s 2850 bytes

because clients only requested needed fields. This becomes important at scale—saving 1KB per request doesn’t sound like much, but multiply by millions of requests and you’re looking at real bandwidth savings.

E. Reliability Under Stress

Figure 7 shows error rates climbing as load increased, but message queues held up remarkably well. At 5000 req/s, REST had an 8.2% error rate, GraphQL hit 12.6%, and RPC saw 5.4%. Message queues? Only 0.8%.

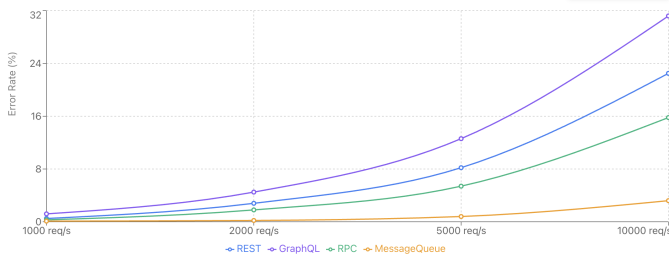


Fig. 7. Error rates under increasing load

This reliability advantage comes from persistent queues that don’t lose messages even when consumers struggle, plus built-in retry mechanisms. If a service goes down briefly, queued messages wait rather than failing immediately.

For mission-critical operations where losing requests isn’t acceptable, this difference is huge.

F. P95 Latency for SLAs

When you’re committing to service level agreements, average latency isn’t enough—you need to know your worst acceptable performance. At 1000 req/s, 95% of RPC requests completed within 132ms. Message queues hit 145ms, REST 168ms, and GraphQL 215ms.

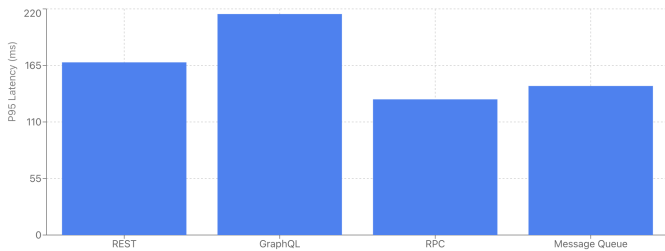


Fig. 8. P95 latency at 1000 req/s load

These P95 numbers matter when you’re promising “99% of requests will complete under 200ms” to customers. RPC and message queues can meet that; GraphQL would struggle.

G. Multi-Dimensional Overview

Figure 9 provides a comprehensive view across five key dimensions. Each protocol shows distinct strengths: REST leads in ease of use, GraphQL in flexibility, RPC in performance, and message queues in scalability and reliability.

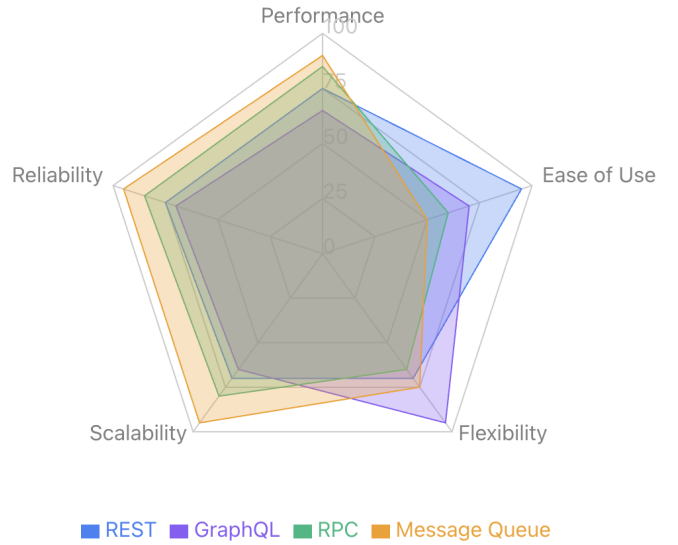


Fig. 9. Multi-dimensional protocol comparison

VI. DISCUSSION

A. What We Learned Building These Systems

Beyond the numbers, actually implementing these protocols taught us about their real-world trade-offs.

REST with FastAPI was genuinely easy. We had endpoints up and running in under an hour, and debugging was straightforward—just look at the HTTP request and response. The main annoyance was over-fetching. Our database queries returned full objects, but clients often need just a few fields. We ended up wasting bandwidth sending unused data.

GraphQL required more upfront thinking. We spent a couple hours designing the schema before writing any resolver code. However, once set up, the flexibility was nice—we could change client data needs without touching the backend. The learning curve was steeper, and debugging resolver chains got tricky when things went wrong. Performance optimization required adding DataLoader to batch database queries, which took some research to implement correctly.

RPC was the most educational because we built it from scratch over RabbitMQ. Implementing correlation IDs to match responses to requests, handling timeouts, setting up reply queues—all of this gave us appreciation for what frameworks like gRPC handle automatically. It worked well once running, but getting there took time. Every error required digging into RabbitMQ management console logs rather than simple HTTP error messages.

Message queues felt cleanest architecturally. Services became truly independent—producers didn’t care who consumed messages or when. But debugging asynchronous flows was harder. When something failed, we couldn’t just trace a single request; we had to look at queue depths, dead letter queues, and consumer logs. Setting up proper monitoring became essential rather than optional.

B. When to Use What

After looking at our results and implementation experiences, here's what we'd recommend:

Choose REST if you're building a public API, need something running quickly, or have a team that's unfamiliar with other options. The tooling is mature, everyone understands it, and it's easy to get right. Accept that you might over-fetch data and need client-side retry logic.

Go with GraphQL when you have diverse clients (web, mobile, different pages) that need different data shapes. The upfront schema work pays off if you're constantly tweaking what data different clients need. Just be ready for higher CPU usage and longer response times under load.

Pick RPC for internal microservice communication where latency matters. The binary protocol saves bandwidth and time. It's great when you control both sides of the communication and can manage the additional complexity. Not recommended for public APIs due to tooling limitations.

Use message queues when you need high throughput, excellent reliability, or can tolerate eventual consistency. They shine for background jobs, event processing, and decoupling services that shouldn't directly depend on each other. Be prepared for operational complexity—you're adding another piece of infrastructure that needs monitoring and maintenance.

C. Limitations and Caveats

Our study has clear boundaries. We tested small systems (2-4 microservices), not the 50+ service architectures some companies run. All testing happened on local machines—we didn't introduce real network latency, packet loss, or the complexities of multi-region deployments.

The load patterns from Artillery were synthetic and predictable. Real user traffic is bursty and unpredictable, which might reveal different failure modes. We also didn't explore protocol combinations, which many production systems use (REST externally, gRPC internally, Kafka for events).

Security testing was minimal—we noted what authentication mechanisms each protocol supports but didn't measure performance impact of encryption or detailed authorization logic.

Think of these results as directional rather than absolute. The relative performance should hold (RPC will generally be faster than REST), but exact numbers will vary based on your infrastructure, workload, and implementation choices.

VII. COMPARISON FRAMEWORK

Table II summarizes everything we learned across five dimensions that matter when choosing a protocol.

VIII. CONCLUSION

We set out to answer a straightforward question: which communication protocol should developers choose for microservices? After building two systems and running extensive tests, our answer is equally straightforward: it depends on what you're optimizing for.

RPC delivered the fastest responses (38ms baseline), making it ideal for latency-sensitive internal communications. Message queues provided the highest throughput (4680 req/s) and best

reliability (0.8% error rate under stress), perfect for high-scale systems that can handle eventual consistency. GraphQL enabled flexible data fetching but demanded more CPU (40% higher usage). REST remained the easiest to implement and understand.

The key insight from our work is that there's no universal winner. Each protocol excels in specific scenarios. The decision framework in Table II maps these trade-offs to real use cases to help developers make informed choices rather than following trends.

For teams building new microservice systems, we recommend starting simple with REST unless you have specific needs that justify additional complexity. Add GraphQL if clients need flexible queries. Introduce RPC when internal latency becomes a bottleneck. Deploy message queues when you need serious scalability or reliability guarantees. And don't be afraid to mix protocols—using REST for external APIs with message queues for internal events is a perfectly reasonable architecture.

IX. FUTURE WORK

Several directions would strengthen this research. Testing larger systems with 10+ microservices would reveal cascading effects and complex interaction patterns we didn't observe in our small setups. Introducing realistic network conditions—latency, packet loss, bandwidth constraints—would show how these protocols perform in actual cloud deployments rather than local containers.

Evaluating hybrid architectures systematically would provide practical guidance since most production systems use multiple protocols. Comprehensive security testing including authentication overhead and encryption impact deserves dedicated study. Cost analysis considering infrastructure expenses, development time, and operational complexity would help make business cases for protocol choices.

Finally, service mesh architectures and their impact on protocol performance is worth investigating as they become more common in production environments.

REFERENCES

- [1] B. Shafabakhsh, R. Lagerström, and S. Hacks, "Evaluating the Impact of Inter Process Communication in Microservice Architectures", in *Proc. QuASoQ @ APSEC*, CEUR-WS, vol. 2767, 2020, pp. 55–63.
- [2] J. Kazanavičius and D. Mažeika, "Evaluation of Microservice Communication While Decomposing Monoliths", *Computing and Informatics*, vol. 42, no. 1, pp. 1–36, May 2023.
- [3] M. Niswar, R. Arisandy Safruddin, A. Bustamin, and I. Aswad, "Performance evaluation of microservices communication with REST, GraphQL, and gRPC", *International Journal of Electronics and Telecommunications*, vol. 70, no. 2, pp. 429–436, 2024.

TABLE II
PRACTICAL COMPARISON FRAMEWORK FOR PROTOCOL SELECTION

Dimension	REST	GraphQL	RPC	Message Queue
Implementation Effort	Very easy with standard HTTP verbs and extensive tooling	Medium due to schema design requirements and resolver implementation	Medium with interface definitions and reply handling complexity	Harder requiring broker setup and consumer/producer patterns
Security	Good support for HTTPS and JWT tokens out of the box	Similar to REST with reliance on HTTPS	Moderate depending on broker's TLS and authentication support	Configurable based on broker configuration
Efficiency	Moderate with HTTP overhead and JSON parsing	Slower due to query parsing and type resolution	Fast with binary serialization and direct communication	Asynchronous with higher per-message latency but excellent batching
Reliability	Moderate requiring client-side retry logic	Moderate with resolver error handling	Good with retry support and guaranteed delivery	High with persistent queues and automatic retries
Best Use Cases	Public APIs, CRUD operations, third-party integrations	Client-driven queries, data aggregation, mobile applications	Internal microservice calls, low-latency requirements, internal APIs	Background processing, event handling, system decoupling